SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1005	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Akhila Pochimi Reddy	**Reg. No.:**	192425225
Date:	27.10.26	**Duration:**	60 minutes

Code of Conduct

I **Akhila PochimiReddy(192425225)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Akhila.P

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description

- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS)	SRS is well-structured, complete, professionally	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or

Documentation (15%)	organized, and includes all required sections.			lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25

1. Study and Analyze the Scenario

1.1 Business Domain

The platform is intended to support the movement of agricultural goods from producers to different categories of buyers through a shared online environment. Instead of relying only on phone calls or in-person visits, users can consult product listings and order information in one place.

A key business principle is that the platform improves access to information; it does not guarantee that every listed product will be available at all times or that a displayed price represents the price across every market. Farmers remain responsible for keeping their listings accurate, while buyers should confirm the order details before purchase.

The Smart Agricultural Marketplace Platform belongs to the agricultural e-commerce domain. It provides a digital platform that connects farmers, wholesalers, retailers, and consumers. Farmers can list

agricultural products, while buyers can view availability, compare prices, place orders, and track deliveries.

The system aims to improve agricultural market access, increase product visibility, simplify transactions, and provide better coordination between agricultural producers and buyers.

1.2 Problem Description

When product information is distributed across different channels, buyers may need to contact several sellers to compare price, quantity, and availability. Farmers may also spend time answering repeated enquiries and coordinating orders manually. These activities can slow down transactions and make it harder to maintain a clear record of agreements.

The proposed system centralizes the main transaction stages: publishing a listing, discovering a product, placing an order, recording payment status, updating delivery progress, and retaining transaction history. This creates a more traceable process for both sides of a transaction.

Farmers often face difficulties reaching buyers directly and obtaining transparent information about market demand and prices. Multiple intermediaries can increase transaction complexity and reduce transparency for both farmers and buyers. Traditional agricultural trading may also involve communication gaps, limited product information, and difficulties tracking orders and deliveries.

The proposed platform provides a centralized digital marketplace for agricultural products, order management, payments, delivery tracking, transaction history, ratings, and market analytics.

1.3 Stakeholders

Each stakeholder has a different information need. Farmers need straightforward listing and stock controls; buyers need clear product details and order status; administrators need oversight and exception handling; and external service providers need a defined way to exchange payment or delivery updates.

The system should show each participant only the information needed for their role. For example, a buyer should be able to see the status and amount of their own order, while administrative access to broader reports should be restricted to authorized accounts.

Stakeholder	Role and Responsibilities

Farmers	Register, list products, update availability, manage orders, and monitor sales.
Wholesalers	Purchase agricultural products in bulk and track orders.
Retailers	Purchase products for resale and manage their orders.
Consumers	Browse products, place orders, make payments, track deliveries, and provide feedback.
Administrator	Manage accounts, monitor transactions, and handle complaints and reports.
Delivery Partners	Update delivery status and provide shipment information.
Payment Gateway	Process payments and communicate transaction status.

1.4 Existing Challenges

These challenges are related to information visibility, coordination, and record keeping. A digital marketplace can reduce repeated communication by making the latest listing and order status visible, but it depends on users entering timely and accurate information.

The platform should therefore provide clear status labels, validation messages, and timestamps. If a product quantity has not been updated recently, the system should avoid presenting the listing as unquestionably current and may prompt the farmer to confirm availability.

· Farmers may have limited direct access to buyers.

· Agricultural prices and product availability may not be sufficiently transparent.

· Multiple intermediaries can complicate transactions.

· Buyers may find it difficult to obtain reliable product and quantity information.

· Order and delivery tracking may require repeated manual communication.

· Maintaining transaction records and collecting customer feedback can be difficult.

· Farmers may lack organized information about market demand and sales performance.

1.5 Objectives

The objectives can be grouped into three areas: marketplace access, transaction management, and decision support. Marketplace access covers registration, listings, search, and availability. Transaction management covers orders, payments, delivery tracking, and history. Decision support covers analytics, ratings, and feedback.

The objectives should be evaluated through demonstrable system behaviour, such as whether a buyer can find an available product, whether an order is stored with the correct total, and whether a user can review the latest recorded transaction status.

1. Register farmers and buyers.
2. Manage agricultural product listings.
3. Display product availability and pricing information.
4. Manage orders from placement to completion.
5. Support secure payments through a payment service.
6. Track deliveries and order status.
7. Maintain transaction histories.
8. Provide market analytics and sales reports.
9. Manage ratings and feedback.
10. Improve price transparency and supply-demand coordination.

2. Functional and Non-Functional Requirements

2.1 Functional Requirements

Requirement implementation should include input validation and clear error handling. For example, a product cannot be listed with a negative quantity, an order cannot be confirmed when the requested

quantity exceeds available stock, and an order total should be calculated by the system rather than relying only on a value typed by the user.

The platform should maintain an understandable status flow for orders, payments, and deliveries. A status change should be recorded with a timestamp and the account or service responsible for the update where appropriate. This supports troubleshooting and helps participants understand what action is required next.

ID	Requirement	Description
FR-01	User Registration	The system shall allow farmers, wholesalers, retailers, and consumers to register.
FR-02	Authentication	The system shall allow registered users to log in and log out securely.
FR-03	Profile Management	Users shall be able to view and update profile information.
FR-04	Product Listing	Farmers shall add products with names, descriptions, quantities, prices, and images.
FR-05	Product Management	Farmers shall update or remove product listings.
FR-06	Product Browsing	Buyers shall browse and search agricultural products.
FR-07	Availability Display	The system shall display available quantities and availability status.
FR-08	Order Placement	Buyers shall select products, specify quantities, and place orders.
FR-09	Order Management	Farmers and buyers shall view and manage relevant order statuses.

FR-10	Payment Processing	The system shall integrate with a payment gateway to process supported payments.
FR-11	Delivery Tracking	Buyers shall view delivery progress and order status updates.
FR-12	Transaction History	The system shall maintain order, payment, and completed transaction records.
FR-13	Market Analytics	The system shall report sales, product demand, and transaction trends from available data.
FR-14	Ratings and Feedback	Buyers shall rate completed purchases and submit feedback.
FR-15	Notifications	The system shall notify users about order, payment, and delivery updates.
FR-16	Administration	The administrator shall manage accounts, monitor transactions, and review reports.

2.2 Non-Functional Requirements

Security and usability should be considered together. The system should protect accounts and transaction information while keeping the most common actions easy to find. Error messages should explain how to correct a problem without exposing passwords, payment credentials, or internal technical details.

Reliability should include recovery from common failures. If a payment service is temporarily unavailable, the system should not incorrectly mark an order as paid. If a notification fails, the underlying order record should remain available so the user can still check the status after signing in.

The proposed performance target should be checked under a stated test setup, including sample data volume, network conditions, and concurrent users. The target is an acceptance goal for the prototype, not a claim that the system has already achieved it.

ID	**Requirement**	**Description**

NFR-01	Performance	Common pages and product searches should respond within approximately 3 seconds under normal conditions.
NFR-02	Security	Use authentication, role-based permissions, encrypted communication, and secure password storage.
NFR-03	Reliability	Preserve valid order and transaction records and handle errors without corrupting data.
NFR-04	Usability	Provide a simple interface suitable for users with different levels of digital literacy.
NFR-05	Scalability	Support increasing numbers of users, products, and orders.
NFR-06	Maintainability	Use modular components so features can be updated independently where practical.
NFR-07	Availability	Remain accessible except during planned maintenance or unexpected interruptions.
NFR-08	Data Integrity	Validate product quantities and order totals and maintain consistent payment statuses.
NFR-09	Compatibility	Work with commonly used modern browsers and mobile devices.
NFR-10	Privacy	Restrict personal and transaction information to authorized users and services.

Note: The numerical response-time target is a proposed design requirement, not a measured result.

3. Identify Actors and Use Cases

3.1 Primary Actors

Actors represent roles rather than individual people; one person may act in different roles only if the system's account and authorization rules allow it. The system should make the active role clear and prevent users from performing operations that are outside their assigned permissions.

Wholesalers and retailers may require quantity-focused purchasing workflows, while consumers may need a simpler browsing and ordering workflow. These differences can be handled through role-specific screens while reusing the same underlying product and order records.

· Farmer: registers, manages product listings, updates quantities and prices, and handles incoming orders.

· Wholesaler: searches for products, places bulk orders, makes payments, and tracks deliveries.

· Retailer: browses products, purchases stock, manages orders, and provides feedback.

· Consumer: searches products, places orders, makes payments, tracks deliveries, and rates purchases.

· Administrator: manages user accounts, monitors transactions, and handles complaints and reports.

3.2 Secondary Actors

External services are dependencies rather than parts fully controlled by the marketplace. Their availability, response formats, fees, and integration rules may change. The system should therefore handle timeouts, rejected requests, and incomplete updates without losing the marketplace's own records.

· Payment Gateway: processes payments and returns payment status.

· Delivery Partner: updates shipment and delivery information.

· Notification Service: sends order, payment, and delivery notifications.

3.3 Major Use Cases

Actor	Use Cases

Farmer	Register, log in, manage profile, add products, update prices, manage stock, view orders, update order status, view sales reports.
Wholesaler	Register, search products, view availability, place bulk orders, make payments, track deliveries, view history, submit feedback.
Retailer	Register, browse products, place orders, make payments, track deliveries, view history, submit ratings.
Consumer	Register, browse products, compare prices, place orders, make payments, track deliveries, submit feedback.
Administrator	Manage accounts, monitor listings and transactions, handle complaints, view analytics, manage reported content.
Payment Gateway	Authorize or reject payments and return transaction status.
Delivery Partner	Receive delivery assignments and update shipment status.
Notification Service	Send notifications about important order and transaction events.

3.4 Use Case Diagram

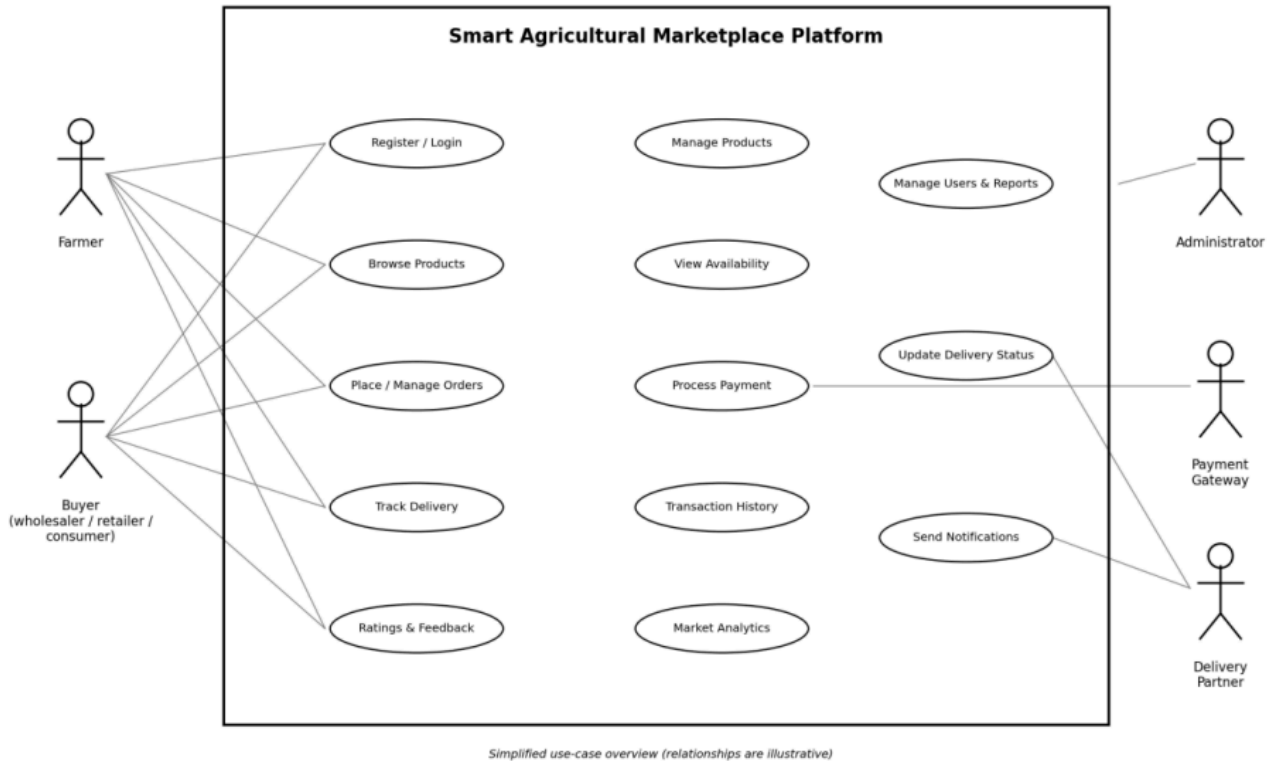


Figure 1: Simplified use-case overview. Relationships are illustrative; a formal UML diagram may redraw these relationships using UML actors, ovals, and a system boundary.

4. Software Requirement Specification (SRS)

4.1 Introduction

This specification is intended to act as a common reference for design, implementation, testing, and evaluation. Each requirement should be reviewed before development and mapped to one or more test cases so that completion can be demonstrated objectively.

The initial system is a proposed academic project. Specific technologies, hosting arrangements, payment providers, delivery partners, and deployment scale are not fixed by the problem statement and should be selected during design.

4.1.1 Purpose

This SRS defines the functional and non-functional requirements of the Smart Agricultural Marketplace Platform. It describes users, features, interfaces, data requirements, and operational limitations.

4.1.2 Product Perspective

The platform is proposed as a centralized web-based marketplace connecting agricultural producers with wholesalers, retailers, and consumers. It integrates product management, order processing, payment services, delivery tracking, transaction records, and analytics.

4.2 Problem Description

Farmers may encounter difficulties connecting directly with buyers and obtaining transparent information about market demand and prices. Existing trading practices can involve several intermediaries, manual communication, and limited visibility into transactions. The platform addresses these issues through a centralized marketplace where farmers can list products, buyers can view availability and place orders, and participants can track order progress and transactions.

4.3 Scope of the System

The first version should prioritize a complete end-to-end transaction path rather than attempting to implement every possible marketplace feature. A suitable prototype path is: register or sign in, publish a product, browse the listing, place an order, record a payment result in a test environment, update delivery status, and review the transaction history.

Features outside the initial scope can be introduced later after the core workflows have been tested. This staged approach reduces implementation risk and makes it easier to demonstrate the system against the assessment requirements.

Included in Scope

- Registration and authentication of farmers and buyers.
- User profile management,Agricultural product listing and updating.
- Product search, browsing, and availability display,Order creation and order-status management.
- Integration with a payment gateway,Delivery-status tracking,Transaction history.
- Ratings and feedback,Market analytics and reporting,Administrative monitoring and reporting.

Outside the Initial Scope

· Direct ownership or operation of farms.

· Physical transportation services operated by the platform itself,Manufacturing or processing agricultural products.

· Guaranteed prediction of future market prices,Ownership of third-party payment or delivery infrastructure.

· Automated financial lending to farmers.

4.4 Functional Requirements

The system shall provide the functions described in Section 2.1. The major feature groups are user and account management; product listing and inventory management; product discovery and availability checking; order creation and processing; payment processing and transaction recording; delivery tracking and notifications; market analytics and reporting; ratings, feedback, and administration.

4.5 Non-Functional Requirements

The system shall meet the quality requirements described in Section 2.2, particularly security, usability, reliability, performance, scalability, maintainability, compatibility, privacy, and data integrity.

4.6 Use Case Descriptions

Across all use cases, the system should validate the user's permissions before changing records. It should also display confirmation after a successful operation and provide a useful message when an operation cannot be completed. The exact wording of messages can be designed during interface development.

For operations that affect money, stock, or order status, repeated submissions should be handled carefully. For example, refreshing a payment result page should not create a duplicate order or incorrectly record the same successful payment twice.

Use Case 1: User Registration

Primary Actor: Farmer, wholesaler, retailer, or consumer.
Precondition: The user can access the registration page.
Main Flow:

1. The user opens the registration page.

2. The user enters the required details.

3. The system validates the information and checks for existing accounts.

4. The system creates the account if validation succeeds.

5. The system displays confirmation.

Alternative Flow: If information is missing or invalid, the system displays an error and requests correction.

Postcondition: A valid user account is created.

Use Case 2: Add Agricultural Product

Primary Actor: Farmer.

Precondition: The farmer is authenticated.

Main Flow:

1. The farmer opens product management.

2. The farmer chooses to add a product.

3. The farmer enters name, description, quantity, price, and other details.

4. The system validates and saves the listing.

5. The listing becomes visible according to its availability status.

Alternative Flow: Invalid information results in validation errors.

Postcondition: The product listing is saved.

Use Case 3: Place an Order

Primary Actor: Buyer.

Precondition: The buyer is authenticated and the requested product is available.

Main Flow:

1. The buyer searches for products and views details.

2. The buyer selects a product and quantity.

3. The system verifies availability and calculates the total.

4. The buyer confirms the order.

5. The system creates the order and records its initial status.

6. The system notifies the relevant farmer.

Alternative Flow: If the quantity is unavailable, the system asks the buyer to revise it.

Postcondition: A valid order is recorded.

Use Case 4: Process Payment

Primary Actor: Buyer. Supporting Actor: Payment Gateway.

Precondition: A valid order exists and is eligible for payment.

Main Flow:

1. The buyer selects a supported payment method.
2. The system sends a payment request to the gateway.
3. The gateway processes the payment and returns the result.
4. The system records the payment status and transaction reference.
5. The system notifies the buyer.

Alternative Flow: If payment fails or is cancelled, the system records the status and informs the buyer. Retry may be allowed.

Postcondition: The payment result is recorded accurately.

Use Case 5: Track Delivery

Primary Actor: Buyer. Supporting Actor: Delivery Partner.

Precondition: The order is accepted and delivery information is available.

Main Flow:

1. The buyer opens order details.
2. The system displays the latest delivery status.
3. The delivery partner updates shipment status as applicable.
4. The system stores the update and informs the buyer.

Alternative Flow: If tracking information is unavailable, the system displays the latest known status or indicates an update is pending.

Postcondition: The buyer can view available delivery progress.

Use Case 6: View Market Analytics

Primary Actor: Farmer or Administrator.

Precondition: The user is authenticated and authorized.

Main Flow:

1. The user opens analytics and selects a report or date range.
2. The system retrieves relevant product, order, and transaction data.
3. The system calculates the requested summary and displays it.

Alternative Flow: If data is insufficient, the system explains that the report cannot yet be generated.

Postcondition: The authorized user views available analytics.

4.7 Data Requirements

The database design should define primary keys, foreign keys, required fields, and validation rules for the listed entities. Related records should be linked through identifiers instead of relying on names alone, because names may be repeated or changed.

Access to personal data should follow the minimum-necessary principle. Reports should expose aggregate information where individual-level details are not required, and sensitive fields should not be included in ordinary logs or notifications.

Entity	Important Attributes
User	User ID, name, contact details, role, account status, password hash.
Farmer Profile	Farmer ID, User ID, farm details, location.
Buyer Profile	Buyer ID, User ID, buyer category, delivery address.
Product	Product ID, farmer ID, name, description, category, price, quantity, availability, image reference.
Order	Order ID, buyer ID, order date, total amount, order status, delivery address.
Order Item	Item ID, order ID, product ID, quantity, unit price, subtotal.
Payment	Payment ID, order ID, amount, payment method, gateway reference, payment status.
Delivery	Delivery ID, order ID, delivery partner reference, tracking reference, status, timestamps.

Transaction	Transaction ID, order ID, payment reference, amount, date, transaction status.
Feedback	Feedback ID, buyer ID, product or order reference, rating, comments, submission date.
Notification	Notification ID, user ID, message type, message, timestamp, delivery status.

4.8 External Interface Requirements

External integrations should define the information sent, the information expected in response, and the actions taken when the response is delayed, rejected, or missing. Credentials and API keys should be stored securely and should not be placed directly in public client-side code.

For an academic demonstration, payment processing should use a sandbox or simulated payment workflow unless a properly configured real payment integration is approved. Test transactions should be clearly separated from real financial transactions.

4.8.1 User Interface

· Registration and login pages, Role-specific dashboards,Product listing and search pages.

· Product details and availability pages, Shopping cart or order confirmation interface.

· Payment status page,Order history and delivery tracking page.

· Ratings and feedback forms,Analytics and administration dashboards.

4.8.2 Software Interfaces

· Database: stores user, product, order, payment, delivery, and feedback information.

· Payment Gateway API: sends payment requests and receives payment results.

· Delivery Service Interface: receives shipment updates when supported by the provider.

· Notification Service: sends notifications through configured communication channels.

4.8.3 Hardware Interfaces

Users may access the system through computers, laptops, tablets, and smartphones with an internet connection. Specialized hardware is not required for basic marketplace operations.

4.8.4 Communication Interfaces

The proposed web application shall use HTTPS for secure client-server communication. External service integrations shall use their supported secure communication protocols.

4.9 System Features

The features should be connected through consistent data and status rules. For example, confirming an order should reserve or re-check stock according to the chosen inventory policy, and a completed delivery should update the order record so that eligible buyers can submit feedback.

Feature 1: User Management

Provides registration, authentication, profile management, and role-based access.

Feature 2: Product and Inventory Management

Allows farmers to create and update listings and maintain availability information.

Feature 3: Marketplace Search

Enables buyers to search products and view prices and available quantities.

Feature 4: Order Management

Supports order creation, confirmation, status updates, and order history.

Feature 5: Payment Management

Integrates with an external payment service and records payment outcomes.

Feature 6: Delivery Tracking

Displays available shipment progress and delivery-status updates.

Feature 7: Analytics and Reporting

Provides summaries of sales, orders, product demand, and transaction trends.

Feature 8: Ratings and Feedback

Allows eligible buyers to submit ratings and comments.

Feature 9: Administration

Supports account oversight, transaction monitoring, issue management, and reporting.

5. Assumptions and Constraints

5.1 Assumptions

The initial release assumes that farmers can maintain their own product details and that buyers will review the information before placing orders. If these assumptions do not hold during user testing, the design may need additional verification steps or administrator support.

The system also assumes that external providers return identifiable transaction or tracking references when those integrations are used. If a provider cannot supply such references, the project must document an alternative procedure.

1. Users have access to a device and internet connection.
2. Farmers and buyers provide valid registration information.
3. Farmers are responsible for accurate product descriptions, prices, and quantities.
4. Buyers review product details before placing orders.
5. A compatible payment gateway is available for integration.
6. Delivery tracking depends on information from the delivery provider.
7. Market analytics are calculated from data recorded by the system.
8. Users have the permissions required for the features they access.
9. The initial implementation focuses on a web-based marketplace.

10. Test data or a sandbox payment environment will be used during development and testing.

5.2 Technical Constraints

The development team should choose tools that are supported by the available laboratory or personal computer environment. The selected database and application framework must support the required relationships, validation, and access controls.

· The system requires a compatible development environment and database.

· Payment processing depends on an external gateway and its API.

· Delivery tracking depends on provider integration or authorized manual updates.

5.3 Operational Constraints

The quality of product availability and market reports depends on timely data entry. A process for correcting inaccurate listings and handling disputed orders should be defined before the system is used beyond a classroom demonstration.

Users may have different levels of digital literacy, so the interface should use familiar labels, simple forms, readable status messages, and a consistent navigation structure.

· Farmers must update product quantities and prices when they change.

· Users must follow account and transaction procedures.

· Delivery estimates may depend on transport providers and local conditions.

· Incorrect user-entered data can affect availability and analytics.

5.4 Economic Constraints

Cost control can be supported by using open-source development tools and limiting paid integrations during the prototype stage. Any later deployment should estimate hosting, backup, security maintenance, support, and transaction-related costs.

The project should distinguish between features that can be demonstrated locally and features that require paid third-party services. This distinction helps set realistic expectations for the academic submission.

· Hosting, database services, domain registration, gateway charges, and maintenance may involve costs.

· The budget may limit access to premium services or advanced analytics.

· The platform should prefer cost-effective tools suitable for an academic prototype.

5.5 Legal and Security Constraints

Before public deployment, the project owner should review applicable requirements for privacy, consumer transactions, payment handling, and record retention. This report identifies these as design constraints and does not substitute for a legal compliance review.

Administrative privileges should be limited to trusted accounts. Important administrative changes, such as disabling accounts or reviewing disputed transactions, should be traceable to support accountability.

· The platform must follow applicable privacy, consumer protection, and electronic transaction requirements.

· Payment information must be handled according to the selected provider's security requirements.

· Personal information shall not be disclosed to unauthorized parties.

· Product descriptions, prices, and transaction records should be handled transparently.

· Applicable refund, dispute, and record-retention rules must be considered before deployment.

5.6 Time Constraints

A practical implementation sequence is requirements confirmation, interface and database design, core listing and ordering, payment and delivery-status handling, analytics and feedback, testing, and documentation. The sequence can be adjusted to match the course schedule.

Testing time should be reserved for negative cases as well as successful flows, including invalid registration data, unavailable stock, rejected payment, missing delivery updates, and unauthorized access attempts.

· The project must be completed within the academic submission schedule.

· Development time must cover analysis, design, implementation, testing, documentation, and demonstration.

· Advanced integrations may be postponed if they cannot be completed within the available time.

6. Validate Requirement Completeness and Consistency

6.1 Stakeholder Requirement Validation

Validation should involve walkthroughs of the main tasks from the perspective of each user role. A farmer should be able to explain how to add and update a listing; a buyer should be able to find a product and place an order; and an administrator should be able to locate a transaction or review a reported issue.

Stakeholder Need	Corresponding Requirement	Validation
Farmers need to list products.	FR-04, FR-05	Verify that farmers can create, update, and remove listings.
Buyers need product information.	FR-06, FR-07	Verify that buyers can view details, prices, and availability.
Users need payment support.	FR-10, FR-12	Test payment outcomes and transaction recording.
Buyers need delivery information.	FR-11, FR-15	Verify delivery updates are displayed and communicated.
Farmers need sales information.	FR-13	Verify reports use recorded and relevant data.

Buyers need feedback.	FR-14	Verify rating submission and eligibility rules.
Administrators need oversight.	FR-16	Verify authorized account and report management.

6.2 Consistency Checks

The same source of truth should be used for product quantity, order total, payment status, and delivery status. Where external providers supply updates, the system should validate the reference and preserve a timestamp of the latest received update.

11. User roles: each actor has defined permissions and responsibilities.

12. Product availability: order placement must verify quantity before confirmation.

13. Payment consistency: a payment must reference a valid order and match the gateway response.

14. Delivery consistency: delivery tracking must reference the corresponding order.

15. Feedback consistency: ratings must follow defined purchase eligibility rules.

16. Analytics consistency: reports must use valid recorded data and must not present estimates as confirmed transactions.

17. Security consistency: sensitive operations require authentication and authorization.

18. Notification consistency: notifications must reflect the latest valid order, payment, or delivery status.

6.3 Ambiguities and Proposed Improvements

Potential Ambiguity	**Proposed Improvement**
Payment methods are not specified.	Define supported methods based on the selected gateway.
Delivery tracking method is not specified.	Decide whether updates use an external API or authorized manual updates.

Product price changes are not defined.	Specify how price changes affect existing orders and confirmed quotations.
Cancellation and refunds are not described.	Define cancellation eligibility, refund rules, and status transitions.
Market analytics are broadly defined.	Specify reports, date ranges, measures, and access permissions.
Ratings and feedback rules are not defined.	Restrict feedback according to a purchase-verification policy.
Bulk-order rules are not specified.	Define minimum quantities, bulk pricing, and quantity limits if required.
Dispute handling is not described.	Establish procedures for complaints, damaged products, and order disagreements.

6.4 Requirement Completeness

Completeness should be reviewed by tracing every objective in the original problem statement to at least one functional requirement and one validation method. Any objective without a corresponding requirement should be added before implementation begins.

The main unresolved decisions are policy choices rather than missing core features: payment methods, cancellation and refund rules, delivery-provider integration, bulk-order pricing, and the exact measures shown in analytics.

The proposed requirements cover the principal functions in the assigned problem statement: registration, product listing, availability display, order management, secure payment integration, delivery tracking, transaction history, analytics, and ratings. External interfaces, data entities, use cases, assumptions, and constraints are also identified.

6.5 Requirement Quality Improvements

A final review should check that each requirement is necessary, unambiguous, feasible, testable, and traceable. Duplicate requirements should be combined, while requirements that contain several independent behaviours should be split into smaller statements.

The project team should keep a simple traceability matrix linking objectives, requirements, use cases, and test cases. This helps demonstrate that the completed prototype addresses the assigned scenario and makes later changes easier to evaluate.

· Give every requirement a unique identifier.

· Make requirements specific, measurable, and testable.

· Avoid vague terms such as 'fast' without defining acceptable performance.

· Review requirements with representatives of intended user groups.

· Maintain traceability between stakeholder needs, system requirements, and test cases.

Conclusion

The Smart Agricultural Marketplace Platform is proposed to connect farmers, wholesalers, retailers, and consumers through a centralized digital marketplace. The requirements analysis identifies stakeholders, challenges, functional and non-functional requirements, actors, use cases, data entities, interfaces, system features, assumptions, and constraints.